



UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

► **Curso: Introducción a la Computación**

► **Docente: Américo Chulluncuy Reynoso**

2026-1

Sesión 6:

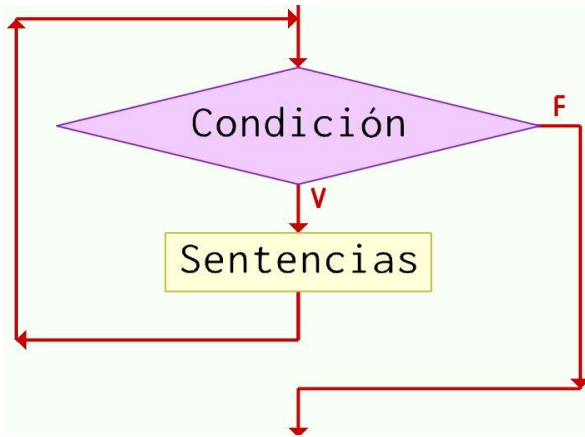
Estructuras de Control: while / for

Contenido

1. **Bucle while en C++**
2. Operadores de incremento (++) y decremento (--)
3. Variables contadoras y acumuladores
4. **Bucle for en C++**
5. Variables centinela para el control de ejecución

1. Bucle while en C++

Bucle: Repite un bloque de código mientras se cumpla una condición.



Sintaxis:

```
while (condición){  
    sentencias;  
}
```



- ✓ Las llaves {} **pueden omitirse si hay una sola sentencia** en el cuerpo del bucle
- ✓ Una **iteración** es una ejecución del cuerpo del bucle
- ✓ **Ejecución:** Se procede a evaluar la condición. Si esta es verdadera, se ejecuta la(s) sentencia(s), luego la condición es evaluada otra vez. Si esta es falsa, el bucle termina.

Ejemplo: bucle while

```
int val = 5;  
while (val >= 0){  
    cout << val << " ";  
    val = val - 1;  
}
```

Salida: 5 4 3 2 1 0

Observaciones

- A diferencia de do-while, while es un bucle de **prueba previa** (la **condición se evalúa antes** de que se ejecute el bucle)
- Si la condición es inicialmente falsa, las sentencias en el cuerpo del bucle nunca se ejecutan.
- Si la condición es inicialmente verdadera, la(s) declaración(es) en el cuerpo continuarán ejecutándose hasta que la condición se vuelva falsa
- El bucle debe contener código que **permita que la condición se vuelva falsa** para salir del bucle. De lo contrario, tendremos un bucle infinito

Ejemplo: while para validación de entrada

```
int edad;

cout << "Ingresa su edad (entre 0 y 120): ";
cin >> edad; // 1. Solicitar y leer datos

// 2. Ingresar al bucle si los datos no son válidos
while (edad < 0 || edad > 120) {
    //3. Muestra mensaje de error
    cout << "Error: edad no válida. Por favor intente de nuevo: ";
    cin >> edad; // vuelva a leer los datos
}

cout << "Edad válida ingresada: " << edad << endl;
```

2. Operadores de incremento (++) y decremento (--)

i++ aumenta (**incrementa**) el valor de la variable i en una unidad
i++; es lo mismo que $i = i + 1$;

j-- reduce (**Decrementa**) el valor de la variable j en una unidad
j--; es lo mismo que $j = j - 1$;

Formas:

i++ Post-incremento: → primero usa el valor, luego incrementa.

++i Pre-incremento: → primero incrementa, luego usa el valor.

Ejemplos

Pre-incremento

```
int x = 1, y = 1;
```

```
x = ++y;
```

```
cout << x << " " << y << endl;  
// imprime 2 2
```

```
x = --y;
```

```
cout << x << " " << y << endl;  
// imprime 1 1
```

Post-incremento

```
int x = 1, y = 1;
```

```
x = y++;
```

```
cout << x << " " << y << endl;  
// imprime 1 2
```

```
x = y--;
```

```
cout << x << " " << y << endl;  
// imprime 2 1
```

3. Variables contadoras, acumuladoras en bucles

Variables contadoras: Permiten realizar un seguimiento de **cuántas veces** ocurre una acción.

```
int contador=0;

while (contador < 10){
    cout << contador + 1 << endl;
    contador++;
}
```

Variables acumuladoras: Se utilizan para **sumar** o **acumular** valores a lo largo de un bucle.

```
int suma = 0;
for (int i = 1; i <= 100; i++) {
    suma += i ; // es equivalente a suma = suma + i
}
cout << "Suma del 1 al 100: " << suma << endl;
```



4. Bucle for en C++

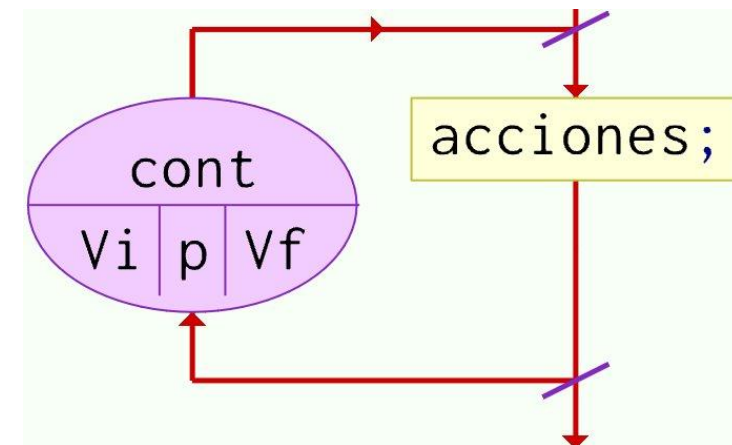
- Bucle de **prueba previa** que se ejecuta cero o más veces. Útil cuando conocemos el número de repeticiones.

Sintaxis

```
for (inicialización; condición; actualización){  
    sentencias;  
}
```

Ejemplo:

```
for (int i = 0; i < 10; i++) {  
    cout << i << " ";  
}
```



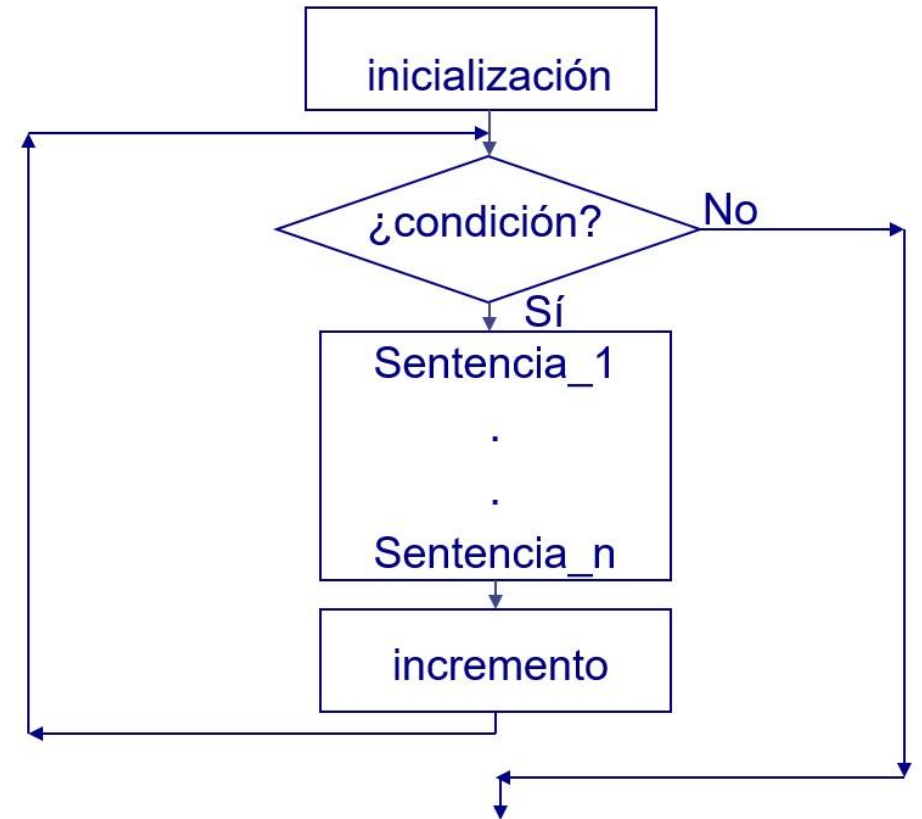
Ejecución del bucle for

Paso 1: Se ejecuta la expresión de **inicialización**

Paso 2: Se evalúa la expresión **condición**, si este es verdadero, vamos al paso 3 en caso contrario se termina el bucle

Paso 3: Se ejecuta el cuerpo del bucle

Paso 4: Se ejecuta la expresión de **actualización (incremento)**. Luego se regresa al Paso 2.



5. Variables centinela para el control de ejecución

Cuando no conocemos de antemano cuántas veces debe repetirse un bucle, usamos una **variable centinela** como condición de salida

```
int valor; // 0 actua como variable centinela
cout << "Ingrese números (0 para salir): ";
cin >> valor;
```

```
while (valor != 0) {
    cout << "Ingresaste: " << valor << endl;
    cin >> valor;
}
```

Ejemplo de aplicación

- En matemática, la sucesión de Fibonacci se trata de una serie infinita de números naturales que empieza con un 0 y un 1 y continúa añadiendo números que son la suma de los dos anteriores:

Algunos de sus términos son:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Escribe un programa que imprima los N primeros términos de la sucesión:

¿Qué bucle usar?

- **do-while**: bucle posterior. El cuerpo siempre se ejecuta al menos una vez. Útil en menús interactivos o validación de entradas
- **while**: Prueba previa. El cuerpo puede no ejecutarse. Útil para validaciones, lectura de datos hasta una condición de corte
- **for**: Prueba previa. Incluye inicialización, condición y actualización en una sola línea. **Útil cuando se conoce el número de repeticiones** o se necesita un contador.

Ejercicios

1. Escribir un programa que permita encontrar todos los triángulos rectángulos de lados enteros (menores a 500) que satisfacen la relación de que la suma de los cuadrados de dos de los lados es igual al cuadrado de la hipotenusa.
2. Escribe un programa que imprima la siguiente forma. Modifique su programa de forma que ahora le pida un número (impar) entre 1 -21 que especifique el número de filas de la figura y muestre la figura de tamaño adecuado
3. Un factorion es un número natural que coincide con la suma de los factoriales de sus dígitos. Escribir un programa que muestre todos los factoriones menores que un numero dado.

```

      *
     ***
    *****
   *********
  ***********
 *****
  *****
   ***
    *
```